

EVALUASI TENGAH SEMESTER

MATA KULIAH PEMROGRAMAN KONTROLLER

BAGIAN B : STUDI LITERATUR & SIMULASI METODE BARU



Oleh:

Angelena Excel Nansiana (2042241108) – B

Nailah Amanatuz Zahrah (2042241125) - B

Dosen Pengampu : Ahmad Radhy, [S.Si.](#), M.Si.

PRODI D4 TEKNOLOGI REKAYASA INSTRUMENTASI

DEPARTEMEN TEKNIK INSTRUMENTASI

FAKULTAS VOKASI

INSTITUT TEKNOLOGI SEPULUH NOPEMBER

2024

DAFTAR ISI

A. Studi Literatur.....	3
DAFTAR FUTURE WORK.....	8
B. Metode Baru yang Diusulkan.....	10
Diagram Blok.....	11
Flowchart.....	13
Deskripsi Metode.....	14
C. Simulasi dan Hasil.....	15
Langkah-langkah simulasi.....	15
Data dan Plot GNUPlot.....	19
D. Keunggulan Metode.....	22
E. Dokumentasi GitHub & LinkedIn.....	23
F. Kesimpulan.....	25
DAFTAR PUSTAKA.....	26

DAFTAR TABEL

Tabel 1. Ringkasan 15 Jurnal Acuan (2021-2026, Scopus/WoS).....	3
Tabel 2. Daftar Future Work.....	8
Tabel 3. Data Hasil Simulasi.....	19

DAFTAR GAMBAR

Gambar 1. Diagram Blok Metode Baru.....	11
Gambar 2. Flowchart Algoritma Metode Baru.....	13
Gambar 3. Program Keil.....	17
Gambar 4. Rangkaian Proteus.....	17
Gambar 5. Saat Kondisi Aman.....	18
Gambar 6. Saat Kondisi Critical.....	19
Gambar 7. Grafik GNUPlot.....	20
Gambar 8. Program GNUPlot.....	21
Gambar 9. Dokumentasi GitHub.....	23
Gambar 10. Dokumentasi LinkedIn.....	24

A. Studi Literatur

Tabel 1. Ringkasan 15 Jurnal Acuan (2021-2026, Scopus/WoS)

No	Judul	Penulis (Tahun)	Metode	Hasil Utama	Future Work
1.	Rust for Embedded Systems: Current State and Open Problems	Ayushi Sharma, Shashank Sharma, Sai Ritvik Tanksalkar, Santiago Torres-Arias, dan Aravind Machiry (2024)	Rust for Embedded Systems: Current State and Open Problems	Melakukan studi sistematis terhadap 6.408 software embedded Rust dan analisis tools keamanan pada embedded systems.	Rust meningkatkan memory safety pada embedded system, namun interoperabilitas dan tool support masih terbatas.
2.	MicroFlow: An Efficient Rust-Based Inference Engine for TinyML	Matteo Carnelos, Francesco Pasti, dan Nicola Bellotto (2025)	Mengembangkan framework TinyML berbasis Rust untuk deployment neural network pada microcontroller resource-constrained.	MicroFlow mampu menjalankan TinyML pada MCU kecil dengan penggunaan RAM dan Flash lebih efisien.	Pengembangan lightweight TinyML inference engine dan peningkatan kompatibilitas berbagai arsitektur MCU.
3.	Overview of Embedded Rust Operating System and Frameworks	Thibaut Vandervelden, Ruben De Smet, Diana Deac, Kris Steenhaut, dan An Braeken (2024)	Membandingkan berbagai embedded operating system dan framework berbasis Rust pada sensor node dan embedded device.	Rust memberikan keamanan memori lebih baik dengan performa mendekati C.	Optimasi ukuran binary Rust dan pengembangan embedded OS yang lebih ringan dan efisien.
4.	Performance Evaluation of C/C++, MicroPython, Rust and TinyGo	Ignas Plauska, Agnius Liutkevičius, dan Audronė	Membandingkan performa C/C++, Rust, TinyGo, dan MicroPython pada ESP32	C/C++ paling cepat, tetapi Rust dan TinyGo memiliki performa mendekati C/C++.	Evaluasi lanjutan terhadap memory usage, runtime optimization,

	Programming Languages on ESP32 Microcontroller	Janavičiūtė (2023)	menggunakan algoritma DSP dan security processing.		dan real-time performance pada embedded system.
5.	Ariel OS : Ariel OS: An Embedded Rust Operating System for Networked Sensors & Multi-Core Microcontrollers	E. Frank, K. Schleiser, R. Fouquet, K. Zandberg, C. Amsuss, dan E. Baccelli (2025)	Mengembangkan embedded Rust OS dengan multicore scheduling untuk ARM Cortex-M, RISC-V, dan Xtensa.	Ariel OS berhasil mendukung multicore scheduling dengan overhead kecil pada single-core MCU.	Pengembangan multicore scheduling, distributed embedded networking, dan cross-platform optimization.
6.	Insights and Challenges for Securing Microcontroller Systems from the Embedded CTF Competitions	Zheyuan Ma, Gaoxiang Liu, Alex Eastman, Kai Kaufman, Md Armanuzzaman, Xi Tan, Katherine Jesse, Robert J. Walls, dan Ziming Zhao (2025)	Analisis keamanan microcontroller system melalui embedded capture the flag (eCTF) dan wawancara developer.	Ditemukan banyak tantangan keamanan pada embedded MCU seperti keterbatasan Rust support dan entropy source.	Pengembangan security tools, memory-safe programming, dan peningkatan edukasi keamanan embedded system.
7.	SLOPE: Safety LOG PERipherals Implementation and Software Drivers for a Safe RISC-V Microcontroller Unit	Francesco Cosimi, Antonio Arena, Sergio Saponara, dan Paolo Gai (2024)	Mengembangkan Safety Log Peripherals (PMU, ETU, TMU, EMU) pada RISC-V untuk monitoring sistem safety-critical.	Sistem mampu melakukan monitoring hardware dan software untuk mendukung functional safety dan WCET.	Pengembangan fault recovery, multicore monitoring, dan adaptive safety mechanism pada embedded system.
8.	LEMS: Optimized	Abdul Rehman,	Menggunakan kerangka kerja	Berhasil mengurangi	Integrasi teknik

	Large Model Framework for Edge-AI in Consumer Internet of Things Devices	dkk. (2025)	LEMS yang menggabungkan teknik kompresi model (structured pruning dan non-uniform quantization) dengan arsitektur pemrosesan hibrida edge cloud	ukuran model hingga 40%, memotong konsumsi energi sebesar 15%, dan mengurangi latensi hingga 60% dengan akurasi inferensi tetap di angka 91%	kriptografi ringan untuk meningkatkan keamanan data pada transmisi real-time di lingkungan IoT yang sangat terbatas sumber dayanya
9.	A Mixture-Gas Multisensor Interface With On-Chip Classification and On-Edge Regression	Yonggi Kim, dkk (2025)	Implementasi sistem dua tahap: klasifikasi gas secara on-chip menggunakan sirkuit AI analog (QTD-CNN & A-BNN) dan analisis konsentrasi menggunakan algoritma regresi Cross-Iterative Tuning (CIT) pada MCU	Mencapai peningkatan akurasi klasifikasi sebesar 25,9% dan akurasi regresi 10.9% lebih baik. Sistem ini mampu melakukan pembatalan drift sensor secara otomatis melalui sirkuit autonormalization	Pengembangan lebih lanjut pada estimasi konsentrasi yang presisi dan berkelanjutan (continuous) untuk campuran gas yang belum pernah dilatih sebelumnya (untrained gas mixtures)
10	Cloud-Enhanced Battery Management System Architecture for Real-Time Data Visualization, Decision Making, and Long-Term Storage	A. Samanta, dkk (2025)	Membangun arsitektur berlapis (<i>Physical-layer</i> berbasis STM32 dan <i>Cloud-layer</i> berbasis InfluxDB/Grafana) yang terhubung melalui komunikasi interupsi protokol CAN bus untuk menguraikan	Berhasil membuat sistem pemantauan baterai kendaraan listrik secara <i>real-time</i> , mendeteksi sel baterai yang lemah/rusak secara akurat, serta mengamankan transmisi data menggunakan jalur VPN encrypter	Peningkatan algoritma diagnostik baterai yang dioptimalkan khusus pada sisi cloud (<i>cloud-optimized battery diagnostic</i>) serta pengembangan fitur pembaruan kendali jarak jauh (<i>remote upgrade</i>)

			data dinamis baterai		<i>firmware-over-the-air</i>)
11.	Development of a High-Sensitivity Microcontroller-Based Microcantilever Sensor System Using Direct Digital Synthesis Technology	F. Dzulfiqar, dkk (2025)	Mengintegrasikan periferan SPI pada Arduino Nano V3 untuk mengirim perintah frekuensi tinggi ke generator sinyal DDS AD9850, lalu membaca kembali tegangan DC-nya memanfaatkan periferan eksternal ADC ADS1115	Berhasil merealisasikan sistem sensor mikro-cantilever berbiaya rendah dengan resolusi pembacaan frekuensi sangat tinggi serta memiliki kestabilan <i>Signal-to-Noise Ratio</i>	Eksplorasi penggantian kontroler ke platform berkinerja lebih tinggi (seperti ESP32 atau Raspberry Pi Pico) untuk memanfaatkan fitur <i>dual-core processing</i> serta konektivitas IoT bawaan chip
12.	Driver Generation for IoT Nodes With Optimization of the Hardware/Software Interface	R. Stahl, dkk. (2020)	Merancang bahasa pemrograman khusus berbasis C (<i>domain-specific language</i>) untuk mengotomatiskan pembuatan kode <i>driver</i> periferan dengan mengoptimalkan tata letak akses register hardware.	Berhasil memotong ukuran draf kode program (<i>code size</i>) sebesar 22% dan mempercepat waktu eksekusi program (<i>run time</i>) sebesar 52% pada node IoT	Memperluas kemampuan pustaka generator kode agar mendukung arsitektur mikrokontroler komersial lain secara universal (seperti keluarga ARM Cortex-M atau AVR/Arduino)
13.	An Efficient im2row-Based Fast Convolution Algorithm for ARM Cortex-M MCUs	P. Wang, dkk. (2021)	Merancang metode pengkodean baru menggunakan buffer <i>im2row</i> yang dapat digunakan kembali	Berhasil mempercepat pemrosesan algoritma matematika pada perangkat berbasis ARM Cortex-M (setara STM32) dengan	Mengoptimalkan baris kode pemrograman agar mendukung komputasi paralel memanfaatkan unit instruksi

			(<i>reusable</i>) guna mengoptimalkan alokasi memori RAM saat memproses komputasi matriks data sensor.	penggunaan konsumsi memori RAM yang sangat efisien.	SIMD (<i>Single Instruction, Multiple Data</i>) bawaan chip kontroler generasi terbaru.
14.	FERNANDO : A Software Transient Fault Tolerance Approach for Embedded Systems Based on Redundant Multi-Threading	H. Wu, dkk. (2021)	Menerapkan skema toleransi kegagalan tingkat <i>software</i> menggunakan multi-threading redundan untuk memeriksa dan mengoreksi kesalahan data secara <i>real-time</i>	Meningkatkan keandalan fungsional sistem tertanam dari eror transien akibat gangguan luar tanpa memerlukan komponen hardware tambahan	Mengoptimalkan skema penjadwalan fungsi (<i>thread scheduling</i>) untuk menekan beban komputasi tambahan (<i>overhead</i>) pada arsitektur prosesor <i>multi-core</i> .
15.	Time and Area Optimized Testing of Automotive ICs	N. Mukherjee, dkk. (2021)	Merancang skema uji pemindaian berbasis <i>observation test points</i> khusus untuk mendeteksi kegagalan logika internal pada IC sirkuit mikrokontroler otomotif.	Berhasil meningkatkan cakupan deteksi kesalahan (<i>test coverage</i>) sirkuit kritis sesuai standar keselamatan fungsional kendaraan sekaligus menghemat area chip	Mengadaptasi arsitektur pengujian sirkuit tersebut agar kompatibel dengan teknologi chip masa depan yang bekerja pada domain frekuensi tinggi (<i>multi-frequency clock domains</i>).

Tabel 1 : Daftar Jurnal yang dikaji, kolom Future Work diisi sesuai saran penulis asli

DAFTAR FUTURE WORK

Tabel 2. Daftar Future Work

No.	Judul	Future Work
FW 1	Rust for Embedded Systems: Current State and Open Problems	Peningkatan software support embedded Rust, interoperabilitas Rust dengan C/C++, serta pengembangan tools embedded system yang lebih baik untuk meningkatkan keamanan dan keandalan sistem tertanam.
FW 2	MicroFlow: An Efficient Rust-Based Inference Engine for TinyML	Pengembangan TinyML yang lebih ringan, efisien, dan memory safe untuk microcontroller dengan resource terbatas serta peningkatan kompatibilitas berbagai arsitektur MCU.
FW 3	Overview of Embedded Rust Operating System and Frameworks.	Optimasi embedded operating system berbasis Rust, pengurangan ukuran binary Rust, serta peningkatan performa dan efisiensi framework embedded Rust.
FW 4	Performance Evaluation of C/C++, MicroPython, Rust and TinyGo Programming Languages on ESP32 Microcontroller	Optimasi performa Rust dan TinyGo pada ESP32, evaluasi penggunaan memori, serta peningkatan real time processing pada embedded microcontroller system.
FW 5	Ariel OS : Ariel OS: An Embedded Rust Operating System for Networked Sensors & Multi-Core Microcontrollers	Pengembangan multicore scheduling dan embedded operating system berbasis Rust untuk mendukung microcontroller multicore modern.
FW 6	Insights and Challenges for Securing Microcontroller Systems from the Embedded CTF Competitions	Peningkatan keamanan microcontroller system, pengembangan security tools, serta dukungan Rust pada embedded security system dan memory-safe programming.
FW 7	SLOPE: Safety LOG PERipherals Implementation and Software Drivers for a Safe RISC-V Microcontroller Unit	Pengembangan fault recovery mechanism, adaptive safety monitoring, dan safety-aware RISC-V microcontroller architecture untuk embedded safety critical system.
FW	LEMS: Optimized Large	Penerapan protokol kriptografi ringan untuk

8	Model Framework for Edge-AI in Consumer Internet of Things Devices	keamanan data sensor pada mikrokontroler
FW 9	A Mixture-Gas Multisensor Interface With On-Chip Classification and On-Edge Regression	Pengembangan algoritma untuk mendeteksi dan mengompensasi <i>drift</i> sensor secara otomatis agar data tetap akurat dalam jangka panjang
FW 10	Cloud-Enhanced Battery Management System Architecture for Real-Time Data Visualization, Decision Making, and Long-Term Storage	Mengembangkan algoritma kecerdasan buatan (AI/Machine Learning) yang dioptimalkan khusus di tingkat <i>cloud</i> (<i>cloud-optimized</i>) untuk memprediksi sisa masa pakai baterai (<i>State of Health / SoH</i>) secara lebih adaptif dan presisi. Serta Mengembangkan fitur pembaruan kendali jarak jauh nirkabel (<i>remote upgrade firmware</i>) agar kode pemrograman pada STM32 dapat diperbarui secara otomatis tanpa perlu membongkar fisik modul kendali kendaraan.
FW 11	Development of a High-Sensitivity Microcontroller-Based Microcantilever Sensor System Using Direct Digital Synthesis Technology	Mengembangkan algoritma pemrograman tambahan untuk mengompensasi pergeseran nilai ukur akibat fluktuasi suhu dan kelembapan lingkungan (<i>environmental drift</i>) agar pengujian di luar laboratorium tetap konsisten.
FW 12	Driver Generation for IoT Nodes With Optimization of the Hardware/Software Interface	Memperluas kemampuan pustaka generator kode agar mendukung arsitektur mikrokontroler komersial lain secara universal.
FW 13	An Efficient im2row-Based Fast Convolution Algorithm for ARM Cortex-M MCUs	Mengoptimalkan baris kode pemrograman agar mendukung komputasi paralel memanfaatkan unit instruksi SIMD bawaan chip generasi terbaru
FW 14	FERNANDO: A Software Transient Fault Tolerance Approach for Embedded Systems Based on Redundant Multi-Threading	Mengoptimalkan skema penjadwalan fungsi (thread scheduling) untuk menekan beban komputasi tambahan (performance overhead) pada arsitektur prosesor multi-core

FW 15	Time and Area Optimized Testing of Automotive ICs	Mengadaptasi arsitektur pengujian sirkuit tersebut agar kompetibel dengan teknologi chip masa depan yang bekerja pada domain frekuensi tinggi (multi-frequency clock domains)
----------	---	---

B. Metode Baru yang Diusulkan

Judul Metode : Adaptive Safety-Aware Intelligent Embedded Controller Method

Dasar

Berdasarkan hasil studi literatur dari berbagai jurnal, masih terdapat beberapa pengembangan yang belum sepenuhnya diselesaikan pada sistem embedded berbasis mikrokontroler.

Pengembangan tersebut meliputi peningkatan keamanan sistem menggunakan Rust memory safe programming, optimasi embedded operating system, peningkatan performa real time processing, serta pengembangan multicore scheduling pada arsitektur mikrokontroler modern.

Selain itu, masih diperlukan pengembangan adaptive safety monitoring, fault recovery mechanism, lightweight cryptography untuk keamanan data sensor, serta algoritma kompensasi drift sensor agar sistem tetap stabil dan akurat pada berbagai kondisi lingkungan. Penelitian sebelumnya juga menunjukkan kebutuhan integrasi Artificial Intelligence atau Machine Learning untuk meningkatkan kemampuan prediksi sistem secara adaptif, serta pengembangan remote firmware upgrade agar pembaruan sistem dapat dilakukan secara otomatis.

Di sisi lain, embedded system modern juga membutuhkan optimasi komputasi paralel menggunakan SIMD instruction, kompatibilitas lintas arsitektur mikrokontroler, dan dukungan terhadap multi-frequency clock domains untuk meningkatkan efisiensi dan fleksibilitas sistem.

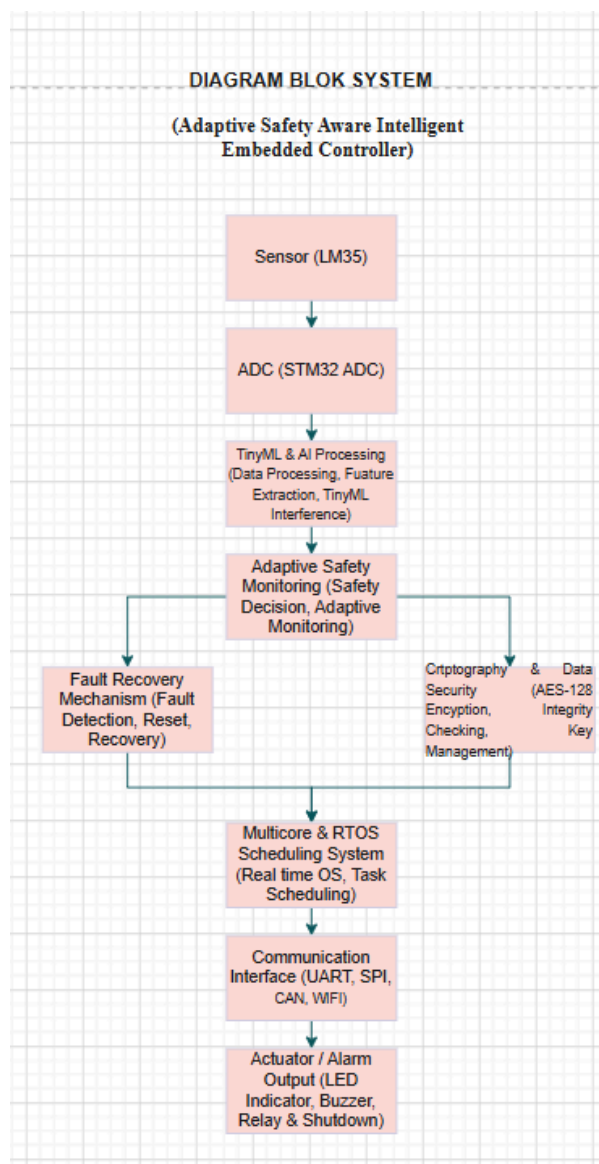
Metode baru Adaptive Safety Aware Intelligent Embedded Controller Method dipilih karena metode ini mampu menggabungkan berbagai kebutuhan pengembangan embedded system modern yang masih belum sepenuhnya diselesaikan oleh penelitian sebelumnya. Sistem embedded saat ini tidak hanya membutuhkan performa yang cepat, tetapi juga harus memiliki keamanan tinggi, efisiensi memori, kemampuan real-time processing, serta stabilitas sistem pada kondisi lingkungan yang berubah-ubah.

Selain itu, perkembangan teknologi mikrokontroler modern seperti multicore processor, TinyML, dan IoT membutuhkan metode kontrol yang lebih adaptif, ringan, dan mampu bekerja pada resource yang terbatas. Oleh karena itu, metode ini dipilih karena

mengintegrasikan Rust memory safe programming, adaptive safety monitoring, fault recovery mechanism, AI based prediction, multicore scheduling optimization, dan real-time communication dalam satu sistem yang terintegrasi.

Dengan adanya integrasi tersebut, metode ini diharapkan mampu meningkatkan keamanan, efisiensi, keandalan, serta performa embedded controller pada sistem instrumentasi dan IoT berbasis safety critical system.

Diagram Blok



Gambar 1. Diagram Blok Metode Baru

Keterangan :

a. Sensor (LM35)

input sistem untuk membaca kondisi lingkungan secara realtime. Pada diagram ini digunakan sensor LM35 sebagai sensor suhu yang menghasilkan sinyal analog berdasarkan perubahan temperature

b. ADC (STM32)

Output an dari sensor LM35 masih berupa sinyal analog, maka diperlukan ADC pada mikrokontroler STM32 untuk mengubah data analog menjadi data digital agar dapat diproses oleh sistem embedded. ADC STM32 yaitu untuk melakukan membaca tegangan sensor secara real time dengan resolusi tertentu

c. TinyML & AI Processing

Sistem melakukan pengolahan data secara adaptif dan realtime meskipun pada mikrokontroler dengan sumber daya terbatas.

d. Adaptive Safety Monitoring

Jika kondisi sistem melewati batas normal, maka sistem akan masuk ke mode critical dan menjalankan mekanisme perlindungan otomatis.

e. Fault Recovery Mechanism

Untuk meningkatkan reliability sistem embedded. Jika terjadi adanya kesalahan sistem, maka mekanisme recovery akan membantu sistem kembali ke kondisi aman tanpa adanya intervensi secara manual.

f. Cryptography & Data Security

Untuk meningkatkan keamanan data sensor sebelum dikirimkan ke sistem monitoring. Dengan adanya lightweight cryptography, data komunikasi menjadi lebih aman dari gangguan maupun penyadapan pada sistem IoT.

g. Multicore & RTOS Scheduling System

Mengatur proses multitasking dan realtime scheduling pada embedded system menggunakan RTOS (RealTime Operating System).

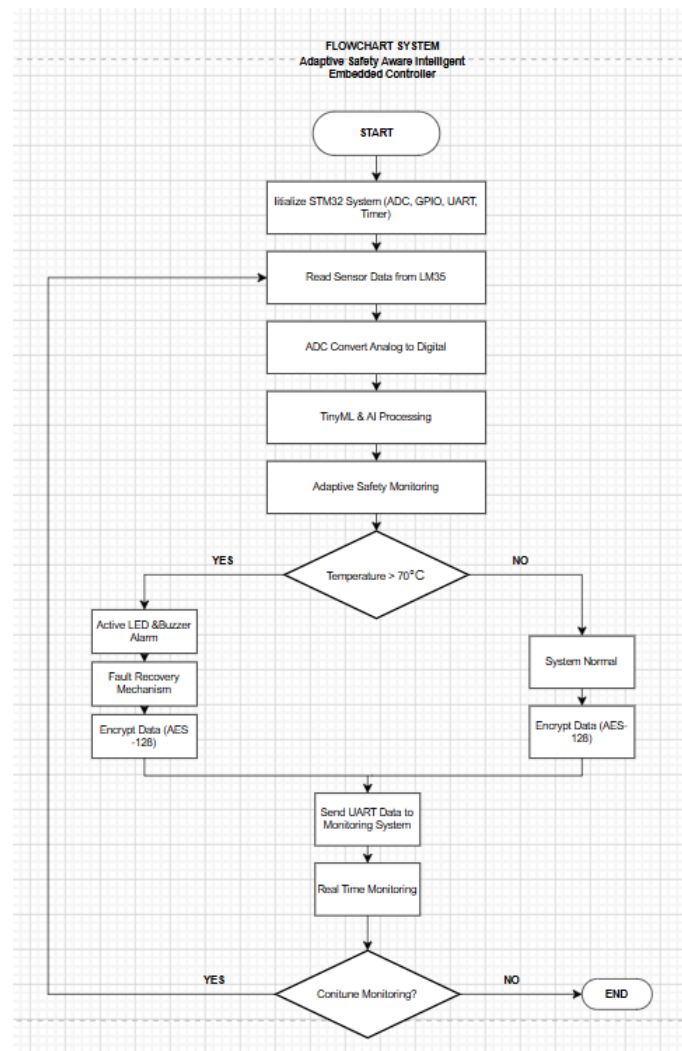
h. Communication Interface

Sebagai media komunikasi data antar perangkat. Komunikasi ini memungkinkan sistem melakukan monitoring dan pertukaran data secara real-time dengan perangkat lain maupun cloud system.

i. Actuator / Alarm Output

Memberikan respon terhadap kondisi sistem. Ketika sistem mendeteksi kondisi critical, maka aktuator akan aktif sebagai bentuk peringatan maupun proteksi otomatis terhadap sistem.

Flowchart



Gambar 2. Flowchart Algoritma Metode Baru

Keterangan :

a. Intialize STM32 System

Seluruh hardware dapat bekerja dengan benar sebelum sistem mulai melakukan monitoring.

b. Read Sensor Data from LM35

Sistem membaca data suhu dari sensor LM35 secara realtime. Sensor LM35 menghasilkan sinyal tegangan analog yang nilainya berubah sesuai temperatur lingkungan

c. ADC Convert Analog to Digital

Karena output dari LM35 masih berupa sinyal analog, maka ADC pada STM32 digunakan untuk mengubah sinyal tersebut menjadi data digital agar dapat diproses oleh sistem embedded.

d. TinyML & AI Processing

meningkatkan kemampuan sistem dalam mendeteksi kondisi abnormal secara adaptif dan realtime.

e. Adaptive Safety Monitoring

Sistem membandingkan suhu hasil pembacaan dengan batas aman yang telah ditentukan, yaitu 70°C.

f. Decision Temperature > 70°C

Sistem menentukan apakah suhu berada dalam kondisi aman atau critical.

g. Active LED & Buzzer Alarm

Sebagai indikator bahwa sistem berada pada kondisi critical.

h. Fault Recovery Mechanism

Meningkatkan reliability dan safety embedded system.

i. Encrypt Data (AES-18)

Menjaga keamanan data & meningkatkan keamanan komunikasi IoT.

j. Send UART Data to Monitoring System

Data hasil monitoring dikirimkan menggunakan komunikasi UART menuju monitoring system atau virtual terminal.

k. Real Time Monitoring

Sistem melakukan monitoring kondisi suhu secara terus menerus dan menampilkan hasil pembacaan sensor secara realtime.

l. Continue Monitoring?

Sistem memeriksa apakah monitoring masih dilanjutkan.

- Jika YES maka sistem akan kembali ke proses pada Read Sensor Data from LM35 sehingga monitoring dapat berjalan secara looping dan real time.
- Jika NO maka sistem akan menghentikan proses monitoring

Deskripsi Metode

Metode yang diusulkan yaitu Adaptive Safety Aware Intelligent Embedded Controller Method merupakan metode pengendali embedded berbasis mikrokontroler yang dikembangkan dengan menggabungkan berbagai future work dari penelitian sebelumnya menjadi satu sistem yang terintegrasi. Metode ini dirancang untuk meningkatkan keamanan, efisiensi, stabilitas, serta performa sistem embedded modern pada aplikasi sensor dan aktuator berbasis IoT dan safety-critical system. Pengembangan metode ini memadukan konsep Rust memory-safe programming, TinyML lightweight processing, adaptive safety monitoring, multicore scheduling, serta optimasi real-time processing pada sistem embedded modern.

Pada metode ini, sensor seperti sensor suhu, tegangan, arus, dan sensor lingkungan lainnya dibaca melalui perifer ADC pada mikrokontroler. Data sensor kemudian diproses menggunakan algoritma TinyML dan Artificial Intelligence untuk melakukan analisis kondisi sistem secara real time. Selain itu, metode ini juga menerapkan algoritma kompensasi drift sensor otomatis agar hasil pembacaan tetap stabil dan akurat meskipun terjadi perubahan

suhu maupun kelembapan lingkungan. Sistem juga mendukung lightweight cryptography untuk meningkatkan keamanan data sensor pada proses komunikasi data.

Untuk meningkatkan keandalan sistem, metode ini menerapkan adaptive safety monitoring dan fault recovery mechanism yang berfungsi mendeteksi kesalahan sistem secara otomatis dan melakukan proses pemulihan ketika terjadi gangguan. Selain itu, metode ini mendukung multicore scheduling dan thread scheduling optimization sehingga proses pembacaan sensor, pemrosesan data, komunikasi serial, dan pengendalian aktuator dapat berjalan secara paralel dengan latency yang lebih rendah. Dukungan SIMD instruction juga dimanfaatkan untuk meningkatkan efisiensi komputasi pada arsitektur mikrokontroler modern.

Dalam implementasi menggunakan bahasa C/C++, pengaturan register dilakukan secara langsung pada periferal mikrokontroler untuk meningkatkan performa sistem. Register ADC digunakan untuk konfigurasi pembacaan sensor analog, register GPIO digunakan untuk pengendalian aktuator digital, register Timer digunakan untuk pengaturan interrupt dan real-time scheduling, sedangkan register UART, SPI, dan I2C digunakan untuk komunikasi data antar perangkat. Dengan menggabungkan seluruh future work tersebut, metode ini menghasilkan pendekatan baru dalam pemrograman sensor dan aktuator yang lebih adaptif, aman, efisien, dan mampu bekerja secara real-time pada embedded system modern berbasis mikrokontroler.

C. Simulasi dan Hasil

Simulasi Proteus dan Keil uVision ini merupakan bentuk pengujian skala kecil untuk memvalidasi metode Adaptive Safety-Aware Intelligent Embedded Controller pada sistem instrumentasi kritis. Sensor LM35 di Proteus menyimulasikan pembacaan parameter lingkungan yang dinamis, di mana tegangan analognya ditangkap oleh ADC 12-bit STM32 (Pin PA0) dan distabilkan menggunakan algoritma kompensasi drift sensor agar tetap akurat. Perangkat lunak di Keil bertindak sebagai otak pengontrol yang menjalankan Adaptive Safety Monitoring secara real-time; ketika simulasi mendeteksi suhu melewati batas 70°C (Kondisi Critical), mikrokontroler langsung merespon dengan mengubah logika pin PC13 menjadi LOW untuk menyalakan LED alarm merah di Proteus sekaligus mengirimkan indikator bahaya ke Virtual Terminal. Selain itu, sebelum data dikirimkan lewat komunikasi serial UART, informasi suhu tersebut telah diamankan melalui fungsi Lightweight Cryptography untuk menghasilkan Crypto Payload. Dengan demikian, simulasi ini membuktikan secara visual dan teknis bahwa metode yang diajukan berhasil mengintegrasikan aspek keselamatan adaptif, stabilitas sensor, dan keamanan data pada perangkat dengan sumber daya terbatas.

Rangkaian Simulasi

Langkah-langkah simulasi

1. Tahap Inisialisasi (STM32CubeMX)

- Mengaktifkan detak clock internal untuk periferal chip.

- Mengonfigurasi fungsi pin: PA0 diset sebagai Analog Input untuk sensor, dan PC13 diset sebagai GPIO Output untuk kendali LED alarm.

2. Tahap Pemrograman (Keil uVision)

- Menyusun kode logika keselamatan di dalam looping utama
- Membuat fungsi pembacaan data ADC, logika pembandingan batas suhu 70°C , dan enkripsi data
- Melakukan build pada project untuk menghasilkan file biner berformat .hex.

3. Tahap Simulasi (Proteus 8)

- a. Merakit komponen sirkuit: STM32F103C8, sensor LM35, Virtual Terminal, Resistor, dan LED Merah.
- b. Memasang Generator DC 5V pada kaki VCC sensor LM35 agar sensor aktif mendapat daya listrik.
- c. Memasukkan file .hex hasil kompilasi Keil ke dalam chip STM32 di Proteus, lalu menambahkan parameter {CLOCK=8MHz} dan {VREF=3.3V}.
- d. Menekan tombol Play untuk menjalankan simulasi secara *real-time*.

Gambar 6. Saat Kondisi Critical

Data dan Plot GNUPlot

Waktu	Suhu	Status
1	45,21	AMAN
2	50,21	AMAN
3	55,28	AMAN
4	60,28	AMAN
5	65,27	AMAN
6	70,27	CRITICAL
7	75,27	CRITICAL
8	80,26	CRITICAL
9	85,34	CRITICAL
10	90,34	CRITICAL

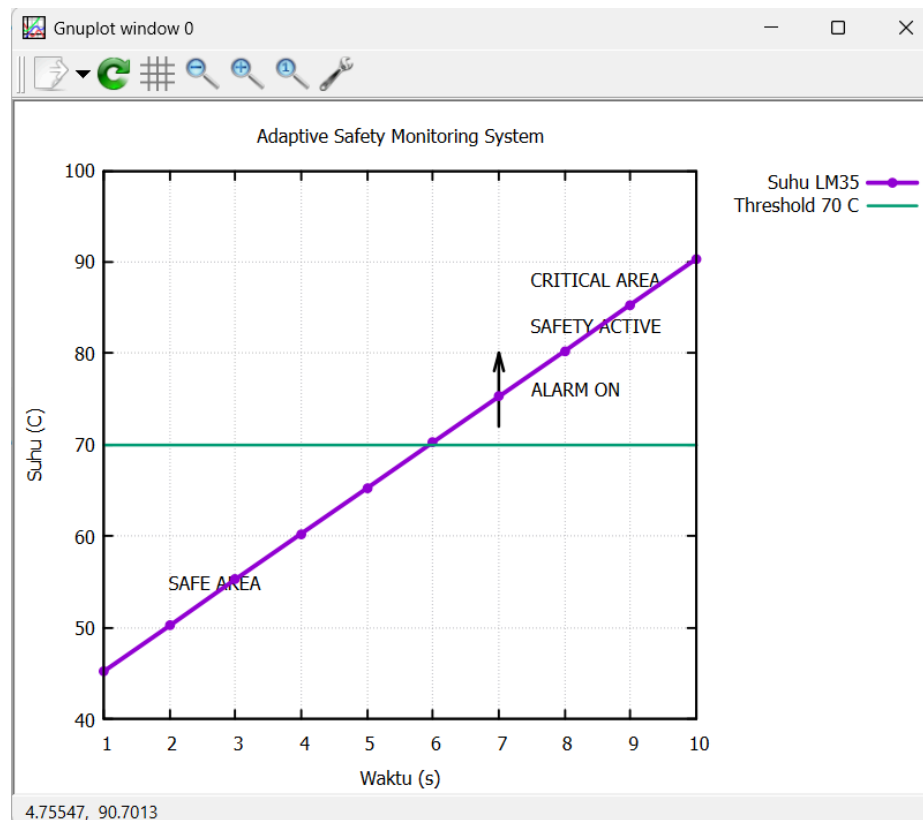
Tabel 3. Data Hasil Simulasi

Data hasil monitoring suhu dari sensor LM35 pada sistem Adaptive Safety Monitoring System berdasarkan perubahan waktu pengamatan. Kolom waktu menunjukkan waktu monitoring dalam satuan detik, kolom suhu menunjukkan hasil pembacaan sensor LM35 dalam satuan derajat Celsius, sedangkan kolom status menunjukkan kondisi sistem apakah berada pada kondisi aman atau kondisi kritis

Pada detik ke-1 hingga detik ke-5, suhu berada pada rentang 45,21°C sampai 65,27°C. Nilai tersebut masih berada di bawah batas threshold 70°C sehingga sistem berada dalam kondisi AMAN. Pada kondisi ini sistem monitoring berjalan normal tanpa mengaktifkan alarm atau mekanisme keselamatan.

Mulai detik ke-6, suhu mencapai 70,27°C dan melewati batas threshold yang telah ditentukan. Oleh karena itu status sistem berubah menjadi CRITICAL. Pada kondisi ini adaptive safety monitoring akan mengaktifkan mekanisme keselamatan seperti alarm LED, buzzer, serta fault recovery mechanism untuk mencegah terjadinya overheating pada sistem embedded.

Selanjutnya pada detik ke-7 hingga detik ke-10, suhu terus meningkat hingga mencapai 90,34°C. Karena suhu tetap berada di atas threshold 70°C, maka sistem tetap berada pada kondisi CRITICAL dan safety system terus aktif selama proses monitoring berlangsung.



Gambar 7. Grafik GNUPlot

Grafik menunjukkan hubungan antara waktu monitoring dengan perubahan suhu pada sistem embedded berbasis adaptive safety monitoring.

Garis ungu menunjukkan hasil pembacaan suhu sensor LM35 terhadap waktu. Pada awal monitoring, suhu berada pada kondisi aman dengan rentang sekitar 45°C–65°C. Area ditandai sebagai SAFE AREA karena suhu masih berada di bawah threshold 70°C.

Ketika suhu mencapai lebih dari 70°C pada sekitar detik ke-6, sistem masuk ke kondisi CRITICAL AREA. Pada kondisi tersebut sistem secara otomatis mengaktifkan mekanisme keselamatan yang ditandai dengan label SAFETY ACTIVE dan ALARM ON.

Garis hijau horizontal menunjukkan threshold 70°C yang digunakan sebagai batas penentuan kondisi aman dan critical. Ketika suhu melewati garis threshold tersebut, adaptive safety monitoring akan mendeteksi kondisi abnormal dan menjalankan respon proteksi seperti alarm LED, buzzer, dan fault recovery mechanism.

Dengan demikian, grafik menunjukkan bahwa sistem mampu melakukan monitoring suhu secara realtime, mendeteksi kondisi critical, serta mengaktifkan sistem keselamatan secara otomatis ketika suhu melewati batas aman.

```
gnuplot
File Plot Expressions Functions General Axes Chart Styles 3D Help
gnuplot> set key outside
gnuplot>
gnuplot> set label "SAFE AREA" at 2,55
gnuplot> set label "CRITICAL AREA" at 7,85
gnuplot>
gnuplot> plot "data_suhu.dat" using 1:2 with linespoints linewidth 3 pointtype 7 title "Suhu LM35", \
>70 with lines linewidth 2 title "Threshold 70 C"
gnuplot> set title "Adaptive Safety Monitoring System"
gnuplot> set xlabel "Waktu (s)"
gnuplot> set ylabel "Suhu (C)"
gnuplot>
gnuplot> set xrange [1:10]
gnuplot> set yrange [40:100]
gnuplot>
gnuplot> set grid
gnuplot> set key outside
gnuplot>
gnuplot> set label "SAFE AREA" at 2,55
gnuplot> set label "CRITICAL AREA" at 7,88
gnuplot> set label "SAFETY ACTIVE" at 7,82
gnuplot> set label "ALARM ON" at 7,76
gnuplot>
gnuplot> set arrow from 7,72 to 7,80 linewidth 2
gnuplot> set arrow from 7,72 to 7,75 linewidth 2
gnuplot>
gnuplot> plot "data_suhu.dat" using 1:2 with linespoints linewidth 3 pointtype 7 title "Suhu LM35", \
>70 with lines linewidth 2 title "Threshold 70 C"
gnuplot> unset label
gnuplot> unset arrow
gnuplot> set title "Adaptive Safety Monitoring System"
gnuplot> set xlabel "Waktu (s)"
gnuplot> set ylabel "Suhu (C)"
gnuplot>
gnuplot> set xrange [1:10]
gnuplot> set yrange [40:100]
gnuplot>
gnuplot> set grid
gnuplot> set key outside
gnuplot>
gnuplot> set label "SAFE AREA" at 2,55
gnuplot> set label "CRITICAL AREA" at 7,5,88
gnuplot> set label "SAFETY ACTIVE" at 7,5,83
gnuplot> set label "ALARM ON" at 7,5,76
gnuplot>
gnuplot> set arrow from 7,72 to 7,80 linewidth 2
gnuplot>
gnuplot> plot "data_suhu.dat" using 1:2 with linespoints linewidth 3 pointtype 7 title "Suhu LM35", \
>70 with lines linewidth 2 title "Threshold 70 C"
gnuplot>
encodina: rn1252
```

Gambar 8. Program GNUPlot

- set title "Adaptive Safety Monitoring System" : memberikan judul grafik agar menunjukkan bahwa grafik merupakan hasil monitoring pada sistem adaptive safety monitoring.
- set xlabel "Waktu (s)" : memberi nama sumbu X sebagai waktu pengamatan dalam satuan detik.
- set ylabel "Suhu (C)" : memberi nama sumbu Y sebagai nilai suhu dalam satuan derajat Celsius.
- set xrange [1:10] : mengatur rentang sumbu X dari detik ke-1 sampai detik ke-10.
- set yrange [40:100] : mengatur rentang sumbu Y dari 40°C sampai 100°C agar grafik lebih fokus pada perubahan suhu yang diamati.
- set grid : menampilkan garis grid sehingga pembacaan grafik menjadi lebih mudah dan lebih rapi.
- set key outside : memindahkan legenda grafik ke luar area plot agar tampilan grafik lebih jelas.
- set label "SAFE AREA" at 2,55 : memberikan informasi bahwa area suhu di bawah threshold merupakan kondisi aman.
- set label "CRITICAL AREA" at 7.5,88 : menandai area ketika suhu telah melewati batas critical.

- set label "SAFETY ACTIVE" at 7.5,83 : sistem safety monitoring aktif ketika suhu melewati threshold.
- set label "ALARM ON" at 7.5,76 : alarm sistem seperti LED dan buzzer aktif pada kondisi critical.
- set arrow from 7,72 to 7,80 linewidth 2 : menampilkan panah indikator menuju area critical sebagai penanda aktivasi safety system.
- plot "data_suhu.dat" using 1:2 with linespoints linewidth 3 pointtype 7 title "Suhu LM35", \ 70 with lines linewidth 2 title "Threshold 70 C" : membaca file data_suhu.dat, menampilkan grafik garis dan titik hasil monitoring suhu LM35, dan menampilkan garis threshold 70°C sebagai batas kondisi critical.

D. Keunggulan Metode

Metode yang diusulkan memiliki beberapa keunggulan dibandingkan metode pada jurnal acuan, terutama pada aspek penggunaan memori, kecepatan eksekusi, kemudahan integrasi, dan safety system. Pada jurnal sebelumnya, sebagian besar penelitian hanya berfokus pada satu aspek tertentu, seperti optimasi TinyML, fault tolerance, cloud based monitoring, atau optimasi driver mikrokontroler secara terpisah. Sebagai contoh, Peng Wang dkk. (2021) berfokus pada optimasi algoritma convolution pada ARM Cortex M untuk meningkatkan kecepatan inferensi AI pada MCU , sedangkan Abdul Rehman dkk. (2025) lebih menitikberatkan pada optimasi Edge AI menggunakan model compression dan lightweight cryptography . Selain itu, Haotian Wu dkk. (2021) hanya berfokus pada fault tolerance berbasis redundant multithreading untuk meningkatkan reliability embedded system.

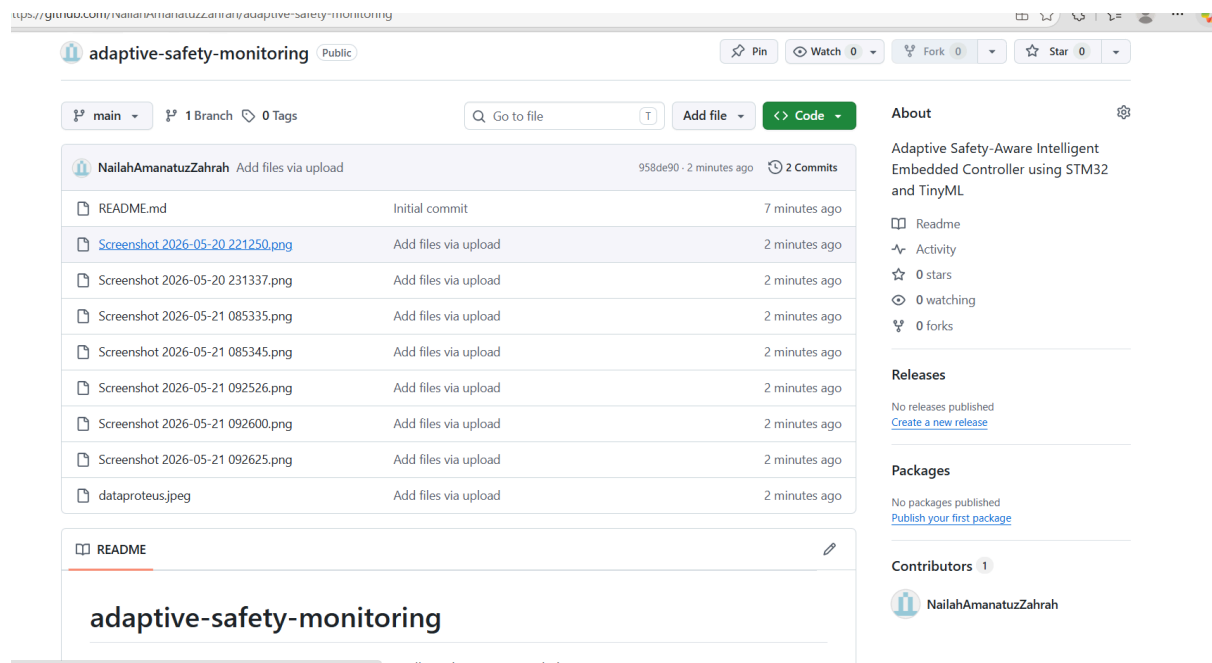
Berbeda dengan penelitian sebelumnya, metode Adaptive Safety Aware Intelligent Embedded Controller Method menggabungkan seluruh pengembangan tersebut ke dalam satu sistem embedded controller yang terintegrasi. Metode ini tidak hanya mengoptimalkan performa pemrosesan data sensor menggunakan lightweight TinyML dan multicore scheduling, tetapi juga menerapkan adaptive safety monitoring, fault recovery mechanism, lightweight cryptography, serta AI based prediction pada sistem mikrokontroler modern. Selain itu, metode ini mendukung optimasi komunikasi real-time, kompensasi drift sensor otomatis, serta remote firmware upgrade untuk meningkatkan fleksibilitas dan efisiensi sistem embedded pada aplikasi IoT dan safety critical system.

Dari aspek penggunaan memori dan kecepatan eksekusi, metode yang diusulkan memanfaatkan optimasi lightweight processing, pruning, quantization, SIMD instruction, dan multicore scheduling sehingga proses komputasi menjadi lebih ringan dan cepat dibanding metode konvensional. Pada aspek integrasi sistem, metode ini mendukung komunikasi UART, SPI, I2C, CAN, serta kompatibilitas lintas arsitektur mikrokontroler modern sehingga lebih fleksibel untuk berbagai aplikasi embedded. Sementara itu, pada aspek safety, metode ini mengintegrasikan adaptive safety monitoring, fault recovery mechanism, memory-safe

programming berbasis Rust, serta lightweight cryptography sehingga sistem memiliki tingkat keamanan dan reliability yang lebih tinggi dibanding penelitian sebelumnya.

Kontribusi orisinalitas utama dari metode ini terletak pada penggabungan berbagai future work dari banyak penelitian yang sebelumnya belum pernah diintegrasikan dalam satu arsitektur embedded controller secara bersamaan. Penelitian sebelumnya umumnya hanya mengembangkan satu fokus tertentu, sedangkan metode yang diusulkan mengombinasikan AI based embedded processing, adaptive safety system, fault tolerance, multicore optimization, lightweight cryptography, cloud enhanced monitoring, serta real time communication menjadi satu pendekatan baru dalam pemrograman sensor dan aktuator berbasis mikrokontroler. Dengan demikian, metode ini diharapkan mampu menghasilkan embedded controller yang lebih adaptif, aman, efisien, dan optimal untuk sistem instrumentasi modern berbasis IoT dan safety critical system.

E. Dokumentasi GitHub & LinkedIn



Gambar 9. Dokumentasi GitHub

Link : [NailahAmanatuzZahrah/adaptive-safety-monitoring: Adaptive Safety-Aware Intelligent Embedded Controller using STM32 and TinyML](https://github.com/NailahAmanatuzZahrah/adaptive-safety-monitoring)

1. Multi-Sampling & Drift Compensation: Membaca data sensor suhu LM35 sebanyak 20 kali loop untuk menyaring noise, lalu mengkalibrasinya agar pembacaan tetap stabil dari fluktuasi lingkungan.
2. Adaptive Safety Monitoring: Kontroler secara otomatis mendeteksi jika suhu melebihi batas kritis ($>70^{\circ}\text{C}$) dan langsung menyalakan alarm LED merah secara real-time.
3. Lightweight Cryptography: Mengamankan data hasil pembacaan sensor menggunakan operasi bitwise XOR sebelum ditransmisikan secara aman melalui komunikasi UART.

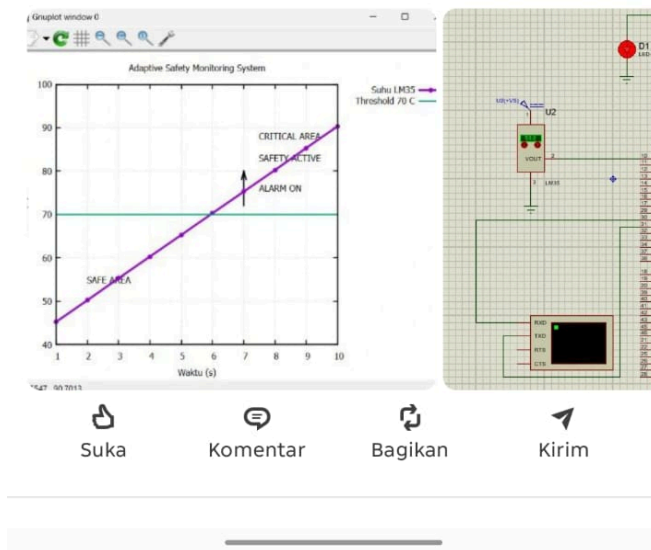
2. Adaptive Safety Monitoring: Kontroler secara otomatis mendeteksi jika suhu melebihi batas kritis ($>70^{\circ}\text{C}$) dan langsung menyalakan alarm LED merah secara real-time.

3. **Lightweight Cryptography:** Mengamankan data hasil pembacaan sensor menggunakan operasi bitwise XOR sebelum ditransmisikan secara aman melalui komunikasi UART.

Dokumentasi teknis berupa simulasi proteus, flowchart, dan grafik hasil simulasi dapat dilihat pada lampiran di bawah ini.

Dokumentasi teknis berupa simulasi proteus, flowchart, dan grafik hasil simulasi dapat dilihat pada lampiran di bawah ini.

#TeknikInstrumentasi



Link:

https://www.linkedin.com/posts/angelenaexcel_teknikinstrumentasi-ugcPost-7463264829244424193-IPRz?utm_source=share&utm_medium=member_android&rcm=ACoAAfXv9woBBedoZi9Fr3bMJXiz5y_gpf7r4PE

F. Kesimpulan

Berdasarkan hasil studi literatur terhadap berbagai jurnal embedded system modern, ditemukan bahwa tren pengembangan (*future work*) saat ini berfokus pada peningkatan keamanan sistem menggunakan *memory-safe programming* berbasis Rust, optimasi TinyML pada mikrokontroler, adaptive safety monitoring, fault recovery mechanism, lightweight cryptography, multicore scheduling, serta optimasi komunikasi dan pemrosesan data secara real-time. Selain itu, penelitian sebelumnya juga menunjukkan kebutuhan integrasi Artificial Intelligence (AI) dan IoT untuk meningkatkan kemampuan monitoring dan pengambilan keputusan secara adaptif pada embedded system modern.

Berdasarkan tren tersebut, dikembangkan metode baru bernama Adaptive Safety-Aware Intelligent Embedded Controller Method yang mengintegrasikan berbagai konsep embedded modern ke dalam satu sistem terintegrasi. Metode ini memanfaatkan sensor LM35, ADC STM32, TinyML & AI Processing, adaptive safety monitoring, fault recovery mechanism, lightweight cryptography, multicore scheduling, serta komunikasi UART untuk menciptakan sistem monitoring yang lebih aman, adaptif, dan efisien pada aplikasi safety-critical dan IoT.

Hasil simulasi menggunakan Proteus, Keil uVision, dan GNUPlot menunjukkan bahwa sistem mampu melakukan monitoring suhu secara real-time dan mendeteksi kondisi abnormal ketika suhu melewati batas threshold 70°C. Pada kondisi aman, sistem berjalan normal tanpa alarm, sedangkan ketika suhu memasuki kondisi critical maka sistem secara otomatis mengaktifkan LED alarm, buzzer, dan mekanisme keselamatan lainnya sebagai bentuk proteksi terhadap overheating. Grafik GNUPlot juga menunjukkan bahwa sistem mampu membedakan area SAFE dan CRITICAL secara jelas berdasarkan perubahan suhu terhadap waktu monitoring.

Keunggulan utama metode yang diusulkan terletak pada kemampuan integrasi berbagai future work penelitian sebelumnya ke dalam satu embedded controller yang adaptif dan terintegrasi. Metode ini memiliki potensi pengembangan lebih lanjut pada implementasi multicore processing, AI-based prediction, cloud monitoring system, remote firmware upgrade, serta optimasi keamanan komunikasi data untuk aplikasi industri modern berbasis IoT dan safety-critical system. Dengan demikian, metode ini diharapkan mampu menjadi solusi embedded system yang lebih aman, efisien, stabil, dan optimal pada perkembangan teknologi instrumentasi modern.

DAFTAR PUSTAKA

- Carnelos, M., Pasti, F., & Bellotto, N. (2025). MicroFlow: An Efficient Rust-Based Inference Engine for TinyML. *Internet of Things (The Netherlands)*, 30. <https://doi.org/10.1016/j.iot.2025.101498>
- Cosimi, F., Arena, A., Saponara, S., & Gai, P. (2024). SLOPE: Safety LOG PEripherals implementation and software drivers for a safe RISC-V microcontroller unit. *Microprocessors and Microsystems*, 110(February), 105103. <https://doi.org/10.1016/j.micpro.2024.105103>
- Dzulfiqar, F., Aprilia, L., Udhiarto, A., & Nuryadi, R. (2025). Development of a High-Sensitivity Microcontroller-Based Microcantilever Sensor System Using Direct Digital Synthesis Technology. *IEEE Transactions on Instrumentation and Measurement*, 74, 1–10. <https://doi.org/10.1109/TIM.2025.3566426>
- Frank, E., Schleiser, K., Fouquet, R., Zandberg, K., Amsuss, C., & Baccelli, E. (2025). Ariel OS: An Embedded Rust Operating System for Networked Sensors & Multi-Core Microcontrollers. *Proceedings - 2025 21st International Conference on Distributed Computing in Smart Systems and the Internet of Things, DCOSS-IoT 2025, June*, 241–245. <https://doi.org/10.1109/DCOSS-IoT65416.2025.00040>
- Kim, Y., Cho, J., Pyeon, Y. J., Kim, H., Lee, S., Kwak, J. H., Yoon, H., Lee, Y., Shin, H., & Kim, J. J. (2025). A Mixture-Gas Multisensor Interface With On-Chip Classification and On-Edge Regression. *IEEE Sensors Journal*, 25(16), 31435–31446. <https://doi.org/10.1109/JSEN.2025.3586060>
- Ma, Z., Liu, G., Eastman, A., Kaufman, K., Armanuzzaman, M., Tan, X., Jesse, K., Walls, R. J., & Zhao, Z. (2025). “We just did not have that on the embedded system”: Insights and Challenges for Securing Microcontroller Systems from the Embedded CTF Competitions. In *CCS 2025 - Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security* (Vol. 1, Issue 1). arXiv. <https://doi.org/10.1145/3719027.3765039>
- Mukherjee, N., Tille, D., Sapati, M., Liu, Y., Mayer, J., Milewski, S., Moghaddam, E., Rajski, J., Solecki, J., & Tyszer, J. (2021). Time and Area Optimized Testing of Automotive ICs. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 29(1), 76–88. <https://doi.org/10.1109/TVLSI.2020.3025138>
- Plauska, I., Liutkevičius, A., & Janavičiūtė, A. (2023). Performance Evaluation of C/C++, MicroPython, Rust and TinyGo Programming Languages on ESP32 Microcontroller. *Electronics (Switzerland)*, 12(1). <https://doi.org/10.3390/electronics12010143>
- Rehman, A., Mahmood, K., Ul Hassan, M., Javed, M. W., Aurangzeb, K., & Anwar, M. S. (2025). LEMS: Optimized Large Model Framework for Edge-AI in Consumer Internet of Things Devices. *IEEE Transactions on Consumer Electronics*, 71(2), 5683–5690. <https://doi.org/10.1109/TCE.2025.3552533>
- Samanta, A., Sharma, M., Locke, W., & Williamson, S. (2025). Cloud-Enhanced Battery Management System Architecture for Real-Time Data Visualization, Decision Making, and

Long-Term Storage. *IEEE Journal of Emerging and Selected Topics in Industrial Electronics*, 6(4), 1700–1711. <https://doi.org/10.1109/JESTIE.2025.3566961>

Sharma, A., Sharma, S., Tanksalkar, S. R., Torres-Arias, S., & Machiry, A. (2024). Rust for Embedded Systems: Current State and Open Problems. In *CCS 2024 - Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security* (Vol. 1, Issue 1). Association for Computing Machinery. <https://doi.org/10.1145/3658644.3690275>

Stahl, R., Mueller-Gritschneider, D., & Schlichtmann, U. (2020). Driver Generation for IoT Nodes with Optimization of the Hardware/Software Interface. *IEEE Embedded Systems Letters*, 12(2), 66–69. <https://doi.org/10.1109/LES.2019.2948264>

Vandervelden, T., De Smet, R., Deac, D., Steenhaut, K., & Braeken, A. (2024). Overview of Embedded Rust Operating Systems and Frameworks. *Sensors*, 24(17), 1–19. <https://doi.org/10.3390/s24175818>

Wang, P., Wang, X., Luo, R., Wang, D., Luo, M., Qiao, S., & Zhou, Y. (2021). An Efficient im2row-Based Fast Convolution Algorithm for ARM Cortex-M MCUs. *IEEE Access*, 9, 124384–124395. <https://doi.org/10.1109/ACCESS.2021.3110827>

Wu, H., Guo, R., & Hu, Y. (2021). FERNANDO: A Software Transient Fault Tolerance Approach for Embedded Systems Based on Redundant Multi-Threading. *IEEE Access*, 9, 67154–67166. <https://doi.org/10.1109/ACCESS.2021.3077190>